

Introduction to Data Engineering & Distributed Architectures

Session 1 of 7 | 4 Hours

Big Data Engineering Programme
Academic Year 2024–2025

Session Overview

Duration	4 hours (theory + lab)
Theory	Data engineering ecosystem, distributed systems
Lab	Set up a local Kafka cluster
Prerequisites	Basic Linux CLI, Python fundamentals
Tools used	Docker, Docker Compose, Apache Kafka

🎯 Learning Objectives

By the end of this session, students will be able to:

- ✔ Define the role of data engineering in a modern big data ecosystem.
- ✔ Distinguish between centralised and distributed system architectures.
- ✔ Explain the core principles of distributed systems (CAP, fault tolerance, scalability).
- ✔ Deploy and verify a local multi-broker Kafka cluster with Docker.
- ✔ Produce and consume messages from a Kafka topic using the CLI.

Contents

1	Part I – Data Engineering in the Big Data Ecosystem	3
1.1	What is Data Engineering?	3
1.2	The Modern Data Stack	3
1.3	Data Engineering vs. Related Roles	3
1.4	The 5 Vs of Big Data	4
1.5	Batch vs. Streaming Workloads	4
2	Part II – Distributed Systems: Architecture & Principles	5
2.1	Why Distributed Systems?	5
2.2	Centralised vs. Distributed Architecture	5
2.3	The CAP Theorem	5
2.4	Core Principles of Distributed Systems	6
2.4.1	Horizontal vs. Vertical Scaling	6
2.4.2	Replication	6
2.4.3	Partitioning (Sharding)	6
2.4.4	Consensus and Leader Election	7
2.4.5	Fault Tolerance and Failure Modes	7
2.5	Messaging Systems in Distributed Architectures	7
3	Part III – Introduction to Apache Kafka	9
3.1	What is Apache Kafka?	9
3.2	Core Kafka Concepts	9
3.3	Kafka Architecture Diagram	10
3.4	Message Retention and Durability	10
4	Part IV – Lab Session	11
4.1	Step 0 – Verify Prerequisites	11
4.2	Step 1 – Create the Project Directory	11
4.3	Step 2 – Write the Docker Compose File	12
4.3.1	2a – File Header and Version	12
4.3.2	2b – First Broker: Identity and Image	12
4.3.3	2c – KRaft Identity Variables	12
4.3.4	2d – Network Listeners	12
4.3.5	2e – Cluster ID and Replication Settings	13
4.3.6	2f – Brokers 2 & 3 (same pattern, different IDs)	14
4.3.7	2g – Kafka UI and Named Volumes	15
4.4	Step 3 – Start the Cluster	16
4.5	Step 4 – Connect to a Broker Container	16
4.6	Step 5 – Create and Inspect a Topic	16
4.6.1	5a – Create the Topic	16
4.6.2	5b – Inspect Partition Leadership	17
4.7	Step 6 – Produce and Consume Messages	17
4.7.1	6a – Terminal 1: Start the Producer	17
4.7.2	6b – Terminal 2: Start the Consumer	18
4.7.3	6c – Consumer Groups	18

4.8	Step 7 – Fault Tolerance: Simulate a Broker Failure	19
4.8.1	7a – Record the Current State	19
4.8.2	7b – Stop Broker 1	19
4.8.3	7c – Observe the Re-election	19
4.8.4	7d – Verify Data is Still Readable	19
4.8.5	7e – Restart Broker 1 and Observe Reintegration	20
4.9	Step 8 – Explore Kafka UI	20
4.10	Step 9 – Teardown	20
5	Summary & Key Takeaways	22
5.1	Vocabulary Checklist	22
5.2	Further Reading	22
5.3	Preview: Session 2	23
	Appendix A – Useful Kafka CLI Commands	24
	Appendix B – Glossary	25

1. Part I – Data Engineering in the Big Data Ecosystem

1.1 What is Data Engineering?

Definition – Data Engineering

Data Engineering is the practice of designing, building, and maintaining the *infrastructure and pipelines* that enable the reliable, scalable, and efficient collection, storage, transformation, and delivery of data for downstream consumers (analysts, data scientists, ML models, and applications).

Data engineers are the **builders of data infrastructure**. Unlike data scientists who consume data to build models, or analysts who query data for insights, data engineers ensure the *plumbing* of the data system is always operational, performant, and trustworthy.

1.2 The Modern Data Stack

The data engineering ecosystem is commonly described as a *Modern Data Stack (MDS)*, encompassing several interconnected layers:

Table 1: Layers of the Modern Data Stack

lightbg Layer	Function	Common Tools
Ingestion	Collect data from sources	Kafka, Airbyte, Fivetran, Debezium
Storage	Persist raw and processed data	HDFS, S3, GCS, Azure Data Lake
Processing	Transform and aggregate data	Spark, Flink, dbt, Beam
Serving	Expose data for consumers	Redshift, BigQuery, Snowflake, APIs
Orchestration	Schedule and coordinate jobs	Airflow, Prefect, Dagster
Observability	Monitor pipelines and quality	Great Expectations, Monte Carlo

1.3 Data Engineering vs. Related Roles

Role Comparison

- **Data Engineer** – builds pipelines, manages infrastructure, ensures data availability.
- **Data Scientist** – analyses data, trains ML models, interprets results.
- **Data Analyst** – queries and visualises data to support business decisions.
- **ML Engineer** – productionises and serves machine learning models at scale.

- **Analytics Engineer** – bridges DE and DA by modelling data with tools like *dbt*.

1.4 The 5 Vs of Big Data

Big data is traditionally characterised by the **5 Vs**:

-  **Volume** – petabytes of data generated daily. and unstructured formats.
-  **Velocity** – real-time or near-real-time data streams.
-  **Veracity** – accuracy and trustworthiness of data.
-  **Variety** – structured, semi-structured,
-  **Value** – the actionable insights derivable from data.

1.5 Batch vs. Streaming Workloads

A fundamental dichotomy in data engineering separates **batch** from **streaming** workloads:

Table 2: Batch vs. Streaming Processing

lightbg Dimension	Batch	Streaming
Data window	Finite, bounded dataset	Unbounded, continuous
Latency	Minutes to hours	Milliseconds to seconds
Throughput	Very high	Lower per second
Complexity	Simpler to reason about	Higher (state, ordering)
Use cases	Nightly reports, ETL jobs	Fraud detection, monitoring
Tools	Spark, Hive, dbt	Kafka, Flink, Spark Streaming

2. Part II – Distributed Systems: Architecture & Principles

2.1 Why Distributed Systems?

A single machine has physical limits on CPU, memory, and disk. Distributed systems overcome these limits by **spreading work across multiple nodes** connected over a network.

Key Motivations

1. **Scalability** – handle workloads beyond a single machine.
2. **Fault Tolerance** – no single point of failure; the system continues despite node failures.
3. **High Availability** – services remain accessible even during partial failures.
4. **Geographic Distribution** – serve users globally with low latency.
5. **Cost Efficiency** – commodity hardware is cheaper than supercomputers.

2.2 Centralised vs. Distributed Architecture

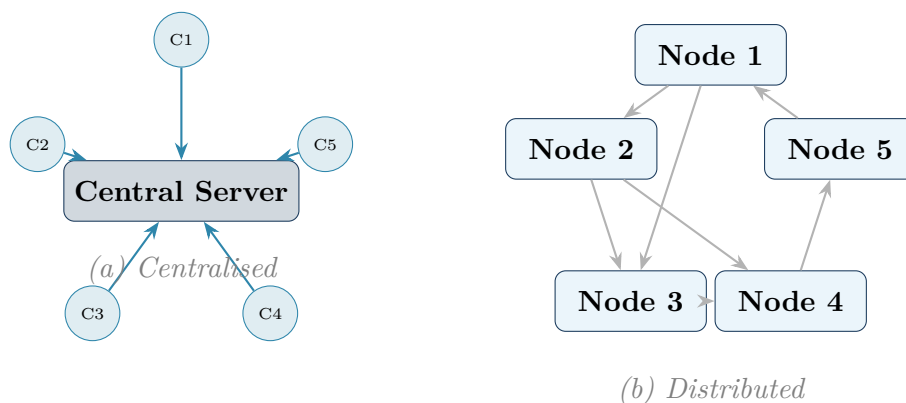


Figure 1: Centralised vs. distributed network topology

2.3 The CAP Theorem

Definition – CAP Theorem (Brewer, 2000)

A distributed system can guarantee at most **two out of three** properties simultaneously:

- **Consistency** – every read receives the most recent write.
- **Availability** – every request receives a response (not necessarily the latest data).
- **Partition Tolerance** – the system continues operating despite network partitions.

Since network partitions *cannot be avoided* in practice, real systems must choose

between **CP** (consistent but possibly unavailable) or **AP** (available but possibly stale).

Table 3: Examples of CAP trade-offs

lightbg System	C	A	P	Category
Apache Kafka	✓	✓	✓	CP / AP (configurable)
Apache Cassandra		✓	✓	AP
HBase	✓		✓	CP
MySQL (single)	✓	✓		CA
Zookeeper	✓		✓	CP

2.4 Core Principles of Distributed Systems

2.4.1 Horizontal vs. Vertical Scaling

- **Vertical scaling** (scale-up): add more CPU/RAM to a single machine. Simple but has physical limits and a single point of failure.
- **Horizontal scaling** (scale-out): add more machines to the cluster. Linear cost growth, better fault tolerance, but requires distributed coordination.

2.4.2 Replication

Definition – Replication

Storing **identical copies of data** on multiple nodes to ensure fault tolerance and high availability. A *replication factor* of r means the system tolerates $r - 1$ node failures without data loss.

Types of replication:

- **Leader-Follower (Master-Replica)** – all writes go to the leader; followers replicate asynchronously or synchronously. Used by Kafka.
- **Multi-Leader** – multiple nodes accept writes; conflicts must be resolved.
- **Leaderless (Quorum-based)** – any node can accept writes; read/write quorums guarantee consistency. Used by Cassandra, DynamoDB.

2.4.3 Partitioning (Sharding)

Definition – Partitioning

Splitting a large dataset into smaller **partitions** spread across multiple nodes, each node being responsible for a subset of the data.

Partitioning strategies:

1. **Range partitioning** – data is divided by ordered key ranges. Simple lookups, but risk of hot spots.
2. **Hash partitioning** – a hash function maps keys to partitions uniformly. Good load distribution, no range queries.
3. **Composite partitioning** – combines range and hash (used by Cassandra).

2.4.4 Consensus and Leader Election

When multiple nodes must agree on a value (e.g., who is the leader), a *consensus algorithm* is required:

- **Paxos** – foundational algorithm; complex to implement.
- **Raft** – simpler, widely adopted (used in etcd, Kafka KRaft).
- **ZAB (Zookeeper Atomic Broadcast)** – used by Apache Zookeeper.

2.4.5 Fault Tolerance and Failure Modes

Common Failure Modes

- **Node crash** – process stops; detectable via heartbeat timeout.
- **Network partition** – nodes cannot communicate; system must decide to sacrifice consistency or availability.
- **Byzantine fault** – node sends incorrect or malicious data; hardest to handle.
- **Slow node** – node responds slowly, causing tail latency.
- **Clock skew** – clocks on different nodes drift; critical for event ordering.

2.5 Messaging Systems in Distributed Architectures

Message queues and *event streaming platforms* are a cornerstone of distributed data engineering:

- They **decouple producers from consumers**, allowing independent scaling.
- They provide **buffering** during traffic spikes.
- They enable **event-driven architectures** (EDA) and microservices.
- They offer **durability**: messages persist even if consumers are temporarily down.

Table 4: Messaging systems comparison

lightbg System	Model	Key Characteristics
Apache Kafka	Log-based pub/sub	High throughput, durable, replayable
RabbitMQ	Queue (AMQP)	Flexible routing, lower throughput
AWS SQS/SNS	Managed queue / fanout	Serverless, AWS-native
Google Pub/Sub	Managed pub/sub	Global, serverless, auto-scaled
Apache Pulsar	Log-based pub/sub	Multi-tenancy, geo-replication

⚠ Common Pitfall

Kafka is not a traditional message queue. Unlike RabbitMQ, Kafka retains messages in an immutable, ordered *log* for a configurable retention period. Consumers *pull* messages at their own pace and can re-read past messages. This is a crucial conceptual distinction.

3. Part III – Introduction to Apache Kafka

3.1 What is Apache Kafka?

Definition – Apache Kafka

Apache Kafka is an open-source **distributed event streaming platform** designed for high-throughput, fault-tolerant, durable, and real-time data pipelines. Originally developed at LinkedIn in 2011 and donated to the Apache Software Foundation, Kafka is today the de facto standard for event-driven data engineering.

3.2 Core Kafka Concepts

Kafka Vocabulary

Event / Record / Message

The fundamental unit of data in Kafka. An event has a *key*, a *value*, a *timestamp*, and optional *headers*.

Topic

A named, ordered, immutable *log* of events. Producers write to topics; consumers read from topics.

Partition

Topics are divided into partitions for parallelism. Each partition is an ordered, append-only sequence of records identified by an *offset*.

Offset

An integer that uniquely identifies a record's position within a partition. Consumers track their position by storing offsets.

Broker

A Kafka server (node). A cluster is composed of multiple brokers for fault tolerance.

Producer

A client application that publishes events to one or more topics.

Consumer

A client application that subscribes to topics and processes events.

Consumer Group

A set of consumers that jointly read from a topic; each partition is assigned to exactly one consumer in the group.

Replication Factor

The number of replicas of each partition across different brokers.

Leader / Follower

For each partition, one broker is the *leader* (handles reads/writes); others are *followers* (replicate the leader).

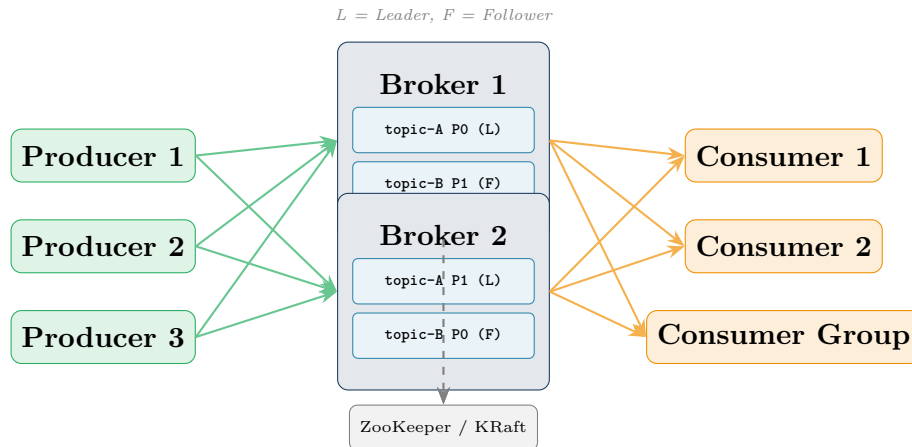
3.3 Kafka Architecture Diagram

Figure 2: High-level Kafka cluster architecture

3.4 Message Retention and Durability

Kafka persists messages to disk. Retention is governed by two policies:

- **Time-based:** retain messages for N hours/days (default: 7 days).
- **Size-based:** retain messages up to X GB per partition.
- **Compaction:** retain only the *latest* record per key (useful for maintaining current state of entities).

4. Part IV – Lab Session

Lab 1 – Set Up a Local Kafka Cluster with Docker

Duration 90 minutes

Objective Deploy a 3-broker Kafka cluster, create topics, produce and consume events, and observe

Tools Docker Desktop, Docker Compose, Kafka CLI (bundled in container)

Difficulty ●● Beginner

Lab Timeline

Time	Activity	Goal
0–10 min	Prerequisites & project setup	Verify tools, create directories
10–25 min	Write <code>docker-compose.yml</code>	Understand KRaft broker config
25–40 min	Start the cluster & verify	Inspect logs, access Kafka UI
40–55 min	Create topics & inspect	Learn partitions and replication
55–75 min	Produce & consume messages	End-to-end message flow
75–90 min	Fault tolerance demo	Observe leader re-election

4.1 Step 0 – Verify Prerequisites

Before writing any configuration, make sure all required tools are available on your machine. Open a terminal and run each command below.

```
docker --version
docker compose version
```

✔ **What to look for:** Docker version 20.10+ and Docker Compose version v2.0+. If either command fails, install Docker Desktop from <https://docs.docker.com/get-docker/>. No Java installation is required — the Kafka CLI is bundled inside the Docker container.

4.2 Step 1 – Create the Project Directory

All files for this lab live in a single dedicated folder.

```
mkdir kafka-lab
cd kafka-lab
touch docker-compose.yml
```

📁 **Why:** Isolating the project in its own folder keeps Docker volume names unique and prevents conflicts with other projects. The only file you will create is `docker-compose.yml` — Kafka stores its data in named Docker volumes, not in local folders.

4.3 Step 2 – Write the Docker Compose File

The `docker-compose.yml` defines three Kafka brokers and one web-based UI. We build it section by section.

4.3.1 2a – File Header and Version

```
version: '3.8'
services:
```

version: '3.8' sets the Docker Compose file format. Version 3.8 supports named volumes, health checks, and all features used here. The **services:** key opens the list of containers we want to run.

4.3.2 2b – First Broker: Identity and Image

```
kafka1:
  image: confluentinc/cp-kafka:7.5.0
  hostname: kafka1
  container_name: kafka1
  ports:
    - "9092:9092"
```

image: uses the official Confluent Platform Kafka image (version 7.5.0, stable). **hostname** and **container_name** both set to `kafka1` so that other containers can reach this broker by name on the Docker internal network. **ports: "9092:9092"** maps the container's external listener to your host machine, so you can connect from outside Docker (e.g. from a Python script running on your laptop).

4.3.3 2c – KRaft Identity Variables

```
environment:
  KAFKA_NODE_ID: 1
  KAFKA_PROCESS_ROLES: 'broker,controller'
  KAFKA_CONTROLLER_QUORUM_VOTERS:
    '1@kafka1:9093,2@kafka2:9093,3@kafka3:9093'
```

KAFKA_NODE_ID: uniquely identifies this broker in the cluster (1, 2, or 3).
KAFKA_PROCESS_ROLES: broker,controller means this node acts both as a *data broker* (stores messages) *and* as a *controller* (participates in leader election). This is the KRaft mode — no ZooKeeper needed.
KAFKA_CONTROLLER_QUORUM_VOTERS lists all nodes that vote in the Raft consensus. The format is `id@host:port`. All three brokers must agree before any leadership change takes effect — this is how Kafka achieves fault tolerance.

4.3.4 2d – Network Listeners

```

KAFKA_LISTENERS:
  'PLAINTEXT://kafka1:29092,
  CONTROLLER://kafka1:9093,
  EXTERNAL://0.0.0.0:9092'
KAFKA_ADVERTISED_LISTENERS:
  'PLAINTEXT://kafka1:29092,
  EXTERNAL://localhost:9092'
KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:
  'CONTROLLER:PLAINTEXT,
  PLAINTEXT:PLAINTEXT,
  EXTERNAL:PLAINTEXT'
KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER

```

Kafka supports **multiple listeners** on different ports for different audiences:

- **PLAINTEXT (port 29092)** — used by brokers to talk to each other *inside* the Docker network (replication, controller communication).
- **CONTROLLER (port 9093)** — dedicated port for the Raft controller-to-controller protocol. Never used by producers or consumers.
- **EXTERNAL (port 9092)** — exposed to your host machine so that external clients (Python scripts, Kafka CLI from your terminal) can connect.

KAFKA_ADVERTISED_LISTENERS tells clients which address to use *after* the initial connection — crucial when Docker NAT translates addresses.

4.3.5 2e – Cluster ID and Replication Settings

```

KAFKA_LOG_DIRS: '/var/lib/kafka/data'
CLUSTER_ID: 'MkU3OEVBNTcwNTJENDM2Qk'
KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 3
KAFKA_DEFAULT_REPLICATION_FACTOR: 3
KAFKA_MIN_INSYNC_REPLICAS: 2
volumes:
  - kafka1_data:/var/lib/kafka/data

```

KAFKA_LOG_DIRS is the path *inside* the container where Kafka writes partition data. Mapped to a named Docker volume so data persists across container restarts.

CLUSTER_ID is a base64-encoded UUID that ties all three brokers together. All brokers in the same cluster *must* share the same **CLUSTER_ID**, otherwise they will refuse to join each other.

KAFKA_DEFAULT_REPLICATION_FACTOR: 3 means every new topic will have 3 replicas by default (one on each broker). No single broker failure can cause data loss.

KAFKA_MIN_INSYNC_REPLICAS: 2 means a write is only acknowledged once at least 2 out of 3 replicas have confirmed it — guaranteeing durability even if one broker crashes during a write.

4.3.6 2f – Brokers 2 & 3 (same pattern, different IDs)

Brokers 2 and 3 follow *exactly* the same structure as Broker 1. Only three values change:

lightbg Variable	Broker 1	Broker 2	Broker 3
KAFKA_NODE_ID	1	2	3
hostname / container_name	kafka1	kafka2	kafka3
External port (host)	9092	9094	9096

Each broker must have a **unique node ID** and a **unique external port** on the host. The internal ports (29092, 9093) are inside the Docker network and do not conflict because each container has its own network namespace.

```
kafka2:
  image: confluentinc/cp-kafka:7.5.0
  hostname: kafka2
  container_name: kafka2
  ports:
    - "9094:9094"
  environment:
    KAFKA_NODE_ID: 2
    KAFKA_PROCESS_ROLES: 'broker,controller'
    KAFKA_CONTROLLER_QUORUM_VOTERS:
      '1@kafka1:9093,2@kafka2:9093,3@kafka3:9093'
    KAFKA_LISTENERS:
      'PLAINTEXT://kafka2:29092,CONTROLLER://kafka2:9093,
      EXTERNAL://0.0.0.0:9094'
    KAFKA_ADVERTISED_LISTENERS:
      'PLAINTEXT://kafka2:29092,EXTERNAL://localhost:9094'
    KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:
      'CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT,EXTERNAL:PLAINTEXT'
    KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
    KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
    KAFKA_LOG_DIRS: '/var/lib/kafka/data'
    CLUSTER_ID: 'MkU30EVBNTcwNTJENDM2Qk'
    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 3
    KAFKA_DEFAULT_REPLICATION_FACTOR: 3
    KAFKA_MIN_INSYNC_REPLICAS: 2
  volumes:
    - kafka2_data:/var/lib/kafka/data

kafka3:
  image: confluentinc/cp-kafka:7.5.0
  hostname: kafka3
  container_name: kafka3
  ports:
    - "9096:9096"
  environment:
    KAFKA_NODE_ID: 3
    KAFKA_PROCESS_ROLES: 'broker,controller'
```



```

KAFKA_CONTROLLER_QUORUM_VOTERS:
  '1@kafka1:9093,2@kafka2:9093,3@kafka3:9093'
KAFKA_LISTENERS:
  'PLAINTEXT://kafka3:29092,CONTROLLER://kafka3:9093,
  EXTERNAL://0.0.0.0:9096'
KAFKA_ADVERTISED_LISTENERS:
  'PLAINTEXT://kafka3:29092,EXTERNAL://localhost:9096'
KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:
  'CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT,EXTERNAL:PLAINTEXT'
KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
KAFKA_LOG_DIRS: '/var/lib/kafka/data'
CLUSTER_ID: 'MkU30EVBNTcwNTJENDM2Qk'
KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 3
KAFKA_DEFAULT_REPLICATION_FACTOR: 3
KAFKA_MIN_INSYNC_REPLICAS: 2
volumes:
  - kafka3_data:/var/lib/kafka/data

```

4.3.7 2g – Kafka UI and Named Volumes

```

kafka-ui:
  image: provectuslabs/kafka-ui:latest
  container_name: kafka-ui
  ports:
    - "8080:8080"
  environment:
    KAFKA_CLUSTERS_0_NAME: local-cluster
    KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS:
      kafka1:29092,kafka2:29092,kafka3:29092
  depends_on:
    - kafka1
    - kafka2
    - kafka3

volumes:
  kafka1_data:
  kafka2_data:
  kafka3_data:

```

Kafka UI is a free, open-source web interface for browsing topics, messages, and consumer groups. It connects to all three brokers via their *internal* PLAINTEXT listeners (port 29092) — it never needs the external ports.

depends_on tells Docker Compose to start the brokers before starting the UI, preventing connection errors on startup.

volumes: at the bottom declares the three named volumes. Docker manages their storage location; they survive `docker compose down` and are only deleted with `docker compose down -v`.

4.4 Step 3 – Start the Cluster

```
docker compose up -d
```

-d (detached) runs containers in the background so your terminal stays free. Docker will pull the images on first run (may take a few minutes depending on your connection). Subsequent starts are instant because images are cached locally.

Wait about 20–30 seconds for all brokers to elect a controller, then verify:

```
docker compose ps
```

All four containers (`kafka1`, `kafka2`, `kafka3`, `kafka-ui`) should show status **Up** or **running**. If a container shows **Exit** or **Restarting**, check its logs with `docker logs kafka1` to identify the error.

To confirm Kafka is fully started, grep the logs for the startup message:

```
docker logs kafka1 2>&1 | grep "started"
```

You should see a line containing `KafkaServer id=1 started`. If the grep returns nothing after 30 seconds, the broker may have failed to join the quorum — usually caused by a wrong `CLUSTER_ID` or a port conflict.

Then open <http://localhost:8080> in your browser. You should see the Kafka UI dashboard showing your cluster named **local-cluster** with 3 brokers online.

4.5 Step 4 – Connect to a Broker Container

All remaining Kafka CLI commands are run *inside* the `kafka1` container, which already has all Kafka tools installed.

```
docker exec -it kafka1 bash
```

docker exec runs a command in an already-running container. **-it** allocates an interactive terminal (you can type and see output). **bash** opens a shell session. From here, all Kafka CLI tools (`kafka-topics`, `kafka-console-producer`, etc.) are available in `/usr/bin/` — no additional installation needed.

4.6 Step 5 – Create and Inspect a Topic

4.6.1 5a – Create the Topic

```
kafka-topics \
  --bootstrap-server kafka1:29092 \
  --create \
  --topic session1-events \
  --partitions 3 \
  --replication-factor 3
```

- bootstrap-server kafka1:29092** is the entry point — Kafka only needs to contact *one* broker to discover the whole cluster.
- partitions 3** creates 3 independent ordered logs. This allows 3 consumer instances to read in parallel (one per partition), multiplying read throughput.
- replication-factor 3** stores a copy of every partition on each of the 3 brokers. The system tolerates 2 broker failures before any data is at risk.

4.6.2 5b – Inspect Partition Leadership

```
kafka-topics \  
  --bootstrap-server kafka1:29092 \  
  --describe \  
  --topic session1-events
```

The output shows one line per partition. For example:

```
Topic: session1-events Partition: 0 Leader: 2 Replicas:  
2,1,3 Isr: 2,1,3
```

Leader: 2 — all reads and writes for Partition 0 go through Broker 2.

Replicas: 2,1,3 — copies exist on all three brokers.

Isr: 2,1,3 — all 3 replicas are currently in-sync (no lag).

Note that Kafka automatically balances leaders across brokers.

4.7 Step 6 – Produce and Consume Messages

Open **two terminal windows**, both connected to the **kafka1** container.

4.7.1 6a – Terminal 1: Start the Producer

```
kafka-console-producer \  
  --bootstrap-server kafka1:29092 \  
  --topic session1-events \  
  --property "parse.key=true" \  
  --property "key.separator="
```

The producer opens an interactive prompt (**>**). Each line you type becomes one Kafka message.

parse.key=true tells Kafka to split each line on the separator to extract a message *key*. Keys are critical: all messages with the same key are always routed to the same partition, guaranteeing order for that key.

key.separator=: uses the colon as the delimiter between key and value.

Now type the following messages and press **Enter** after each:

```
user1:{"event":"login","ts":"2024-01-01T10:00:00Z"}  
user2:{"event":"purchase","item":"laptop","amount":1299}  
user1:{"event":"logout","ts":"2024-01-01T10:05:00Z"}
```

Both `user1` messages share the same key, so Kafka routes them to the *same partition* in the *same order*. This guarantees that a consumer reading that partition will always see `login` before `logout` for `user1` — a critical property for event-driven systems.

4.7.2 6b – Terminal 2: Start the Consumer

In a **new terminal**, open a second shell in the container:

```
docker exec -it kafka1 bash

kafka-console-consumer \
  --bootstrap-server kafka1:29092 \
  --topic session1-events \
  --from-beginning \
  --property "print.key=true" \
  --property "print.offset=true" \
  --property "print.partition=true"
```

`--from-beginning` replays all messages from offset 0. Without this flag, the consumer would only read messages produced *after* it connects — useful in production but not for exploration. `print.partition` and `print.offset` display routing metadata alongside each message. You will notice that the two `user1` events appear on the same partition, while `user2` may land on a different one.

4.7.3 6c – Consumer Groups

Stop the previous consumer (`Ctrl+C`), then run it again with a group name:

```
kafka-console-consumer \
  --bootstrap-server kafka1:29092 \
  --topic session1-events \
  --group analytics-group \
  --from-beginning
```

`--group analytics-group` registers this consumer as part of a named group. Kafka assigns partitions to group members and *tracks the offset* for the group. The next time a consumer in `analytics-group` connects, it will only receive messages it has not yet processed — enabling reliable at-least-once delivery.

Check the consumer group lag (offset committed vs. latest):

```
kafka-consumer-groups \
  --bootstrap-server kafka1:29092 \
  --describe \
  --group analytics-group
```

The **LAG** column shows how many messages are waiting to be read. A lag of 0 means the consumer is fully caught up. A growing lag indicates the consumer is too slow — a key metric to monitor in production pipelines.

4.8 Step 7 – Fault Tolerance: Simulate a Broker Failure

This is the most important exercise of the lab. We will crash one broker and observe that Kafka automatically recovers.

4.8.1 7a – Record the Current State

Before stopping anything, run the describe command and write down which broker is the *Leader* for each partition:

```
kafka-topics \
  --bootstrap-server kafka1:29092 \
  --describe \
  --topic session1-events
```

4.8.2 7b – Stop Broker 1

In a **new terminal on your host machine** (outside any container):

```
docker stop kafka1
```

docker stop sends a SIGTERM signal to the container process and waits for a graceful shutdown. This simulates a server crash or planned maintenance.

4.8.3 7c – Observe the Re-election

Back inside `kafka2`, re-run the describe command:

```
kafka-topics \
  --bootstrap-server kafka2:29092 \
  --describe \
  --topic session1-events
```

Within a few seconds, Kafka's Raft controller will have elected a new leader for every partition that was led by Broker 1. The ISR column will now show only 2 replicas (Broker 2 and Broker 3) since Broker 1 is offline.

Key insight: the cluster is still fully operational with 2 out of 3 brokers. The `MIN_INSYNC_REPLICAS: 2` setting we configured means producers and consumers continue working without interruption.

4.8.4 7d – Verify Data is Still Readable

```
kafka-console-consumer \
  --bootstrap-server kafka2:29092 \
  --topic session1-events \
  --from-beginning
```

All previously produced messages are still accessible. Because we used `replication-factor 3`, every message exists on at least two surviving brokers. This is the fundamental value proposition of Kafka replication.

4.8.5 7e – Restart Broker 1 and Observe Reintegration

```
docker start kafka1
```

After a few seconds, Broker 1 will:

1. Reconnect to the Raft quorum and fetch missing log entries from the current leader.
2. Catch up on any messages produced while it was offline (*log replication*).
3. Rejoin the ISR once fully synchronised — you will see it reappear in the ISR column of the `-describe` output.

It may or may not reclaim its previous leadership roles; Kafka does not automatically rebalance leaders after recovery unless you explicitly trigger it.

4.9 Step 8 – Explore Kafka UI

Open <http://localhost:8080> and navigate through the following sections to reinforce the concepts covered in the lab:

lightbg UI Section	What to Look For
Brokers	List of 3 nodes; which one is the active controller
Topics	<code>session1-events</code> with 3 partitions; replication map
Messages	Browse individual records; filter by partition or offset
Consumer Groups	<code>analytics-group</code> ; lag per partition
Schema Registry	Empty for now — will be used in Session 4

4.10 Step 9 – Teardown

```
# Stop containers but keep the data volumes
docker compose down
```

```
# Full cleanup: stop containers AND delete all data
docker compose down -v
```

docker compose down stops and removes containers, but named volumes survive — your messages will still be there when you run **docker compose up** again.

docker compose down -v also deletes the volumes. Use this to start completely fresh, e.g. before the next lab session.

Reflection Questions

Answer these questions before the next session:

1. You stopped Broker 1. Which brokers became the new leaders for each partition? Does this match your prediction based on the Replicas list?
2. What would happen if you had set `replication-factor 1` before stopping the only broker holding a partition?

3. How does `MIN_INSYNC_REPLICAS: 2` protect against data loss? What happens if a third broker also goes down while Broker 1 is still stopped?
4. Why does Kafka use *key-based routing* for partitions rather than random routing for all messages?
5. What is the difference between *consumer offset tracking* and simply re-reading from `-from-beginning` every time?

5. Summary & Key Takeaways

📌 What We Covered Today

1. **Data engineering** sits at the intersection of software engineering and data management; it builds the reliable infrastructure that powers data products.
2. The **Modern Data Stack** is layered: ingestion → storage → processing → serving, each layer with specialised tools.
3. Distributed systems solve scalability and fault tolerance at the cost of complexity: network partitions, clock skew, and consensus are inherent challenges.
4. The **CAP theorem** forces a design choice between consistency and availability when a network partition occurs.
5. **Apache Kafka** is a distributed, log-based event streaming platform: topics are partitioned and replicated across brokers.
6. **Replication** and **ISR** ensure Kafka remains durable and available even when brokers fail.
7. In the lab, we deployed a 3-broker cluster, created topics, produced/consumed events, and observed automatic leader re-election.

5.1 Vocabulary Checklist

Before the next session, make sure you can define and use the following terms:

- | | |
|------------------------------------|--|
| ✓ Distributed system | ✓ Message queue vs. event log |
| ✓ CAP theorem | ✓ Kafka broker / topic / partition |
| ✓ Horizontal vs. vertical scaling | ✓ Offset |
| ✓ Replication & replication factor | ✓ Producer / consumer / consumer group |
| ✓ Partitioning / sharding | ✓ ISR (In-Sync Replicas) |
| ✓ Consensus algorithm | ✓ Leader election |

5.2 Further Reading

- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly. Chapters 5–6 (Replication, Partitioning).
- Apache Kafka Documentation: kafka.apache.org/documentation
- Confluent Kafka Fundamentals: developer.confluent.io
- Brewer, E. A. (2000). *Towards robust distributed systems*. PODC Keynote.

- Gilbert, S. & Lynch, N. (2002). *Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services*. ACM SIGACT.

5.3 Preview: Session 2

Next session we go deeper into Kafka internals:

- Producers: partitioning strategies, acks, idempotence, batching.
- Consumers: offset management, commit strategies, consumer group rebalancing.
- Message durability: `acks`, `min.insync.replicas`, `retries`.
- Lab: Build a complete Kafka streaming pipeline in Python.

Appendix A – Useful Kafka CLI Commands

```
1 #           Topic Management
2 kafka-topics --bootstrap-server <host:port> --list
3 kafka-topics --bootstrap-server <host:port> --create \
4   --topic <name> --partitions <n> --replication-factor <r>
5 kafka-topics --bootstrap-server <host:port> --describe --topic <name>
6 kafka-topics --bootstrap-server <host:port> --delete --topic <name>
7
8 #           Producer
9 kafka-console-producer --bootstrap-server <host:port> --topic <name>
10
11 #           Consumer
12 kafka-console-consumer --bootstrap-server <host:port> \
13   --topic <name> --from-beginning
14 kafka-console-consumer --bootstrap-server <host:port> \
15   --topic <name> --group <group-id>
16
17 #           Consumer Groups
18 kafka-consumer-groups --bootstrap-server <host:port> --list
19 kafka-consumer-groups --bootstrap-server <host:port> \
20   --describe --group <group-id>
21 kafka-consumer-groups --bootstrap-server <host:port> \
22   --reset-offsets --to-earliest \
23   --topic <name> --group <group-id> --execute
24
25 #           Config
26 kafka-configs --bootstrap-server <host:port> \
27   --describe --entity-type brokers --entity-name 1
```

Listing 1: Kafka CLI Quick Reference

Appendix B – Glossary

lightbg Term	Definition
Broker	A Kafka server process that stores and serves partitions.
Consumer Group	A logical group of consumers sharing topic consumption.
Docker Compose	A tool for defining multi-container applications in YAML.
ETL	Extract-Transform-Load: a data pipeline pattern.
Fault Tolerance	System's ability to continue operating despite failures.
ISR	In-Sync Replica: follower that is fully caught up with the leader.
KRaft	Kafka Raft – the built-in consensus mechanism replacing ZooKeeper.
Log compaction	Keeping only the latest record per key in a partition.
Offset	Unique sequential integer identifying a record in a partition.
Partition	An ordered, immutable sub-sequence of a Kafka topic.
Producer	A client that writes events to Kafka topics.
Replication Factor	Number of partition copies across brokers (≥ 1).
Topic	A named, categorised sequence of events in Kafka.
ZooKeeper	Legacy distributed coordination service; replaced by KRaft in Kafka 3+.